LaTeX to Markdown Converter Report

Shuvam Chakraborty
Entry No. - 2024MCS2004

August 25, 2024

Contents

0.1 Introduction

The LaTeX to Markdown Converter project is a tool designed to convert LaTeX documents into Markdown format. This conversion facilitates the transition from LaTeX's complex typesetting system to Markdown's more straightforward syntax, which is commonly used for web content and documentation.

0.1.1 Objectives

The main goal of this project is to develop a converter that translates LaTeX files into Markdown, maintaining the document's original structure and formatting as closely as possible. This tool aims to make LaTeX documents accessible in a format that is easier to work with for web content and simpler documentation tasks.

0.1.2 Scope

The project encompasses:

- Tokenization of LaTeX commands using Lex.
- Construction of an Abstract Syntax Tree (AST) with Bison.
- Generation of Markdown output from the AST.
- Automated build and execution using Makefile and scripts.
- Comprehensive documentation with Doxygen.
- Unit testing and code coverage using GTest and lcov.

0.1.3 Motivation

The motivation behind this project is to enable users to leverage Markdown's simplicity while retaining the rich formatting capabilities of LaTeX. By converting LaTeX documents to Markdown, users can more easily integrate their content into web-based platforms and documentation tools.

0.1.4 Background

LaTeX is a high-quality typesetting system often used in academia for technical and scientific documents. Markdown is a lightweight markup language popular for web content and documentation. Converting LaTeX to Markdown allows users to transition between these formats, benefiting from Markdown's ease of use while preserving the core content and formatting of LaTeX documents.

0.2 Project Overview

This project is a LaTeX to Markdown converter built using Lex and Bison. It reads LaTeX files, generates an Abstract Syntax Tree (AST), writes the AST to a file, and then traverses the AST to produce the Markdown output, preserving the structure and formatting as closely as possible.

0.3 Project Structure

The Latex2Markdown repository is organized into several components:

- **README.md**: Provides an overview and documentation of the project.
- **parser**: Contains core files and scripts for processing LaTeX files:
  - **Doxyfile**: Configuration file for generating documentation with Doxygen.
  - **doc/**: Directory for storing generated documentation.
  - **input.tex**: Sample LaTeX input file for testing the parser.
  - **latexmarkdown.l**: Lex file for tokenizing LaTeX commands.
  - **latexmarkdown.h**: Header file defining structures and functions for the parser.
  - **latexmarkdown.y**: Bison file for generating the AST and Markdown output.
  - **Makefile**: Automates the build and execution process.
  - **document.sh**: Script for generating documentation.
  - **run.sh**: Script to simplify running the code.
- **testing**: Contains files and scripts for unit testing:
  - **CMakeLists.txt**: Configuration file for setting up the build system with CMake.
  - **coverage/**: Directory for storing code coverage reports.
  - **test_latexmarkdown.cpp**: Unit tests implemented using GTest.
  - **build/**: Directory for building the testing components.
  - **latexmarkdown.h**: Header file for testing.
  - **build_and_test.sh**: Script to build and run the tests.
  - **read_coverage_report.sh**: Script to read and display the coverage report.

0.4 Features

0.4.1 Lexer Tokenization

The Lex file is responsible for scanning and tokenizing various LaTeX commands. It identifies key elements such as sections, text formatting, images, lists, tables, and custom commands, preparing them for further processing.

0.4.2 Bison-based AST Creation

The Bison file constructs an Abstract Syntax Tree (AST) from the tokenized LaTeX input. It processes the tree to generate Markdown output, converting LaTeX structures into their Markdown equivalents while preserving the document's formatting.

0.4.3 Automated Build and Execution

The project includes a Makefile and a run.sh script to streamline the build, execution, and output generation processes. These tools simplify compiling the parser, running it, and checking the outputs.

0.4.4 Comprehensive Documentation

Doxygen is utilized to generate detailed documentation for the project. It covers all modules, functions, and features, ensuring that the codebase is well-documented and easy to understand.

0.4.5 Unit Testing and Code Coverage

Unit tests are implemented using GTest, and code coverage is verified with lcov. This approach ensures that the code is thoroughly tested and meets quality standards, providing confidence in its functionality and robustness.

0.5 Workflow

The workflow involves several key steps:

- Tokenization: Lex processes the LaTeX input to produce tokens.
- Parsing: Bison uses these tokens to create an Abstract Syntax Tree (AST).
- Conversion: The AST is traversed to generate the Markdown output, which is saved to output.md.
- Documentation and Testing: The build and test scripts are used to manage the project lifecycle, including documentation generation and code testing.

0.6 Usage

0.6.1 Clone GitHub Repository

To clone the repository, use the following command:

```
git clone https://github.com/Shuvam-Chakraborty/Latex2Markdown.git
```

0.6.2 Build and Run the Parser

Navigate to the parser directory:

```
cd ~/Latex2Markdown/parser
```

Compile and run the converter:

```
make
```

Clean up all intermediate files:

```
make clean
```

View the output files:

```
cat ast.tex
cat output.md
```

Clean up output files:

```
make outclean
```

Compile and run the converter using the script:

```
cd ~/Latex2Markdown/parser
./run.sh input.tex output.md
```

View the output files:

```
cat ast.tex
cat output.md
```

Clean up output files:

```
make outclean
```

0.6.3 Create Documentation

Generate project documentation:

```
cd ~/Latex2Markdown/parser
make
./document.sh
make clean
```

0.6.4 Run Unit Tests and Check Coverage

Run unit tests and check code coverage:

```
cd ~/Latex2Markdown/testing
./build_and_test.sh
./read_coverage_report.sh
```

0.7 Contributing

Contributions are welcome! To contribute:

- Create issues or submit pull requests on the GitHub repository.
- Follow the contribution guidelines outlined in the repository.

0.8 Appendices

0.8.1 Code Listings

Sample Snippet of Lex File

The following snippet shows the Lex file (‘latexmarkdown.l‘) used for tokenizing LaTeX commands:

```
%{
#include "latexmarkdown.tab.h"
#include <string.h>
%}

%option noyywrap

%%

\\documentclass[^\n]+ {
    yylval.str = strdup(yytext);
    return DOCCLASS;
}

\\usepackage[^\n]+ {
    yylval.str = strdup(yytext);
    return USP;
}

\\title[^\n]+ {
    yylval.str = strdup(yytext);
    return TITLE;
}

\\author[^\n]+ {
    yylval.str = strdup(yytext);
    return AUTHOR;
}

\\date[^\n]+ {
    yylval.str = strdup(yytext);
    return DATE;
}

\\begin\{document\} {
    yylval.str = strdup(yytext);
    return BEGINDOC;
}

\\end\{document\} {
    yylval.str = strdup(yytext);
```

```
    return ENDDOC;
}

\\textbf\{[^\}]*\} {
    yylval.str = strdup(yytext + 8);
    yylval.str[strlen(yylval.str) - 1] = '\0';
    return BOLD;
}

\\begin\{itemize\} {
    yylval.str = strdup(yytext);
    return BEGIN_ITEMIZE;
}

\\end\{itemize\} {
    yylval.str = strdup(yytext);
    return END_ITEMIZE;
}

[^\n]+ {
    yylval.str = strdup(yytext);
    return TEXT;
}

\n+ {
    yylval.str = strdup(yytext);
    return NEWLINE;
}

%%
```

Sample Snippet of Header File for Converter

The following snippet shows the Header File for Converter:

```
#ifndef LATEXMARKDOWN_H
#define LATEXMARKDOWN_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

typedef struct astnode {
    char *token;
    char *data;
    int tabs;
    int unvisited;
```

```
    struct astnode *children[30];
    int child_count;
} astnode;

astnode* createNode(char*, char*, int );
void addChild(astnode*, astnode*);
int shouldSkip(const char*);
void printAST(astnode*, FILE*);
void freeAST(astnode*);
void process_item(char*, FILE*);
void yyerror(const char*);
int yylex(void);

astnode* createNode(char *token, char *data, int tabs) {
    astnode *node = (astnode*)malloc(sizeof(astnode));
    if (node == NULL) {
        fprintf(stderr, "Error: memory allocation failed\n");
        exit(1);
    }
    node->token = strdup(token);
    node->data = strdup(data);
    node->tabs = tabs;
    node->unvisited = 1;
    node->child_count = 0;
    for (int i = 0; i < 30; i++) {
        node->children[i] = NULL;
    }
    return node;
}

void addChild(astnode *parent, astnode *child) {
    if (parent == NULL) {
        fprintf(stderr, "Error: null parent pointer\n");
        return;
    }
    if (child == NULL) return;
    if (parent->child_count < 30) {
        parent->children[parent->child_count] = child;
        parent->child_count++;
    } else {
        fprintf(stderr, "Warning: Node '%s' exceeded maximum children limit
    }
}

int shouldSkip(const char *token) {
    const char *skipList[] = { "usepac_list", "sections_list", "subsection
    int skipCount = sizeof(skipList) / sizeof(skipList[0]);
    for (int i = 0; i < skipCount; i++) {
```

```
        if (strcmp(token, skipList[i]) == 0) {
            return 1;
        }
    }
    return 0;
}

void printAST(astnode *node, FILE *file) {
    if (node == NULL) return;
    if (!shouldSkip(node->token)) {
        for (int i = 0; i < node->tabs; i++) {
            fprintf(file, "--");
        }
        fprintf(file, "%s:-%s\n", node->token, node->data);
    }
    for (int i = 0; i < node->child_count; i++) {
        printAST(node->children[i], file);
    }
}

void freeAST(astnode *node) {
    if (node == NULL) return;
    for (int i = 0; i < node->child_count; i++) {
        freeAST(node->children[i]);
    }
    free(node);
}

void process_item(char *str, FILE *file) {
    while (isspace((unsigned char)*str) || (unsigned char)*str == '\t') {
        str++;
    }
    if (strncmp(str, "\\item", 5) == 0) {
        str += 5;
    }
    while (isspace((unsigned char)*str)) {
        str++;
    }
    char *result_str = strdup(str);
    if (result_str == NULL) {
        fprintf(stderr, "Memory-allocation-failed\n");
        return;
    }
    fprintf(file, "%s", result_str);
    free(result_str);
}

void yyerror(const char *s) {
```

```
    fprintf(stderr, "Error: %s\n", s);
}

#endif /* LATEXMARKDOWN_H */
```

Sample Snippet of Bison Code

The following snippet shows the Bison file ('latexmarkdown.y') used for parsing:

```
%{
#include "latexmarkdown.h"
astnode *root;
%}

%union {
    char *str;
    struct astnode *node;
}

%token <str> DOCCLASS USP TITLE AUTHOR DATE BEGINDOC ENDDOC SECTION SUBSECT

%type <node> document preamble documentclass usepac_list usepackage title a

%start document

%%

document: preamble body
        {
            $$ = createNode("document", "", 0);
            root = $$;
            addChild($$, $1);
            addChild($$, $2);
        }
        ;

preamble: documentclass usepackage title author date
        {
            $$ = createNode("preamble", "", 1); /**< Create a "preamble" n
            addChild($$, $1);
            addChild($$, $2);
            addChild($$, $3);
            addChild($$, $4);
            addChild($$, $5);
        }
        ;

bold: BOLD NEWLINE
    {
```

```
            $$ = createNode("bold", "", 7);
            astnode* temp1 = createNode("bold_text", $1, 8);
            addChild($$, temp1);
            astnode* temp2 = createNode("newline", $2, 8);
            addChild($$, temp2);
        }
        ;

itemize: BEGIN_ITEMIZE NEWLINE itemize_body END_ITEMIZE NEWLINE
        {
            $$ = createNode("itemize", "", 7);
            astnode* temp1 = createNode("start_itemize", "", 8);
            addChild($$, temp1);
            astnode* temp2 = createNode("newline", $2, 8);
            addChild($$, temp2);
            addChild($$, $3);
            astnode* temp3 = createNode("end_itemize", "", 8);
            addChild($$, temp3);
            astnode* temp4 = createNode("newline", $5, 8);
            addChild($$, temp4);
        }
        ;

%%
```

0.8.2 Sample Files

Input File: `input.tex`

```
\documentclass{article}
\usepackage{graphicx} % Required for inserting images
\usepackage{hyperref}
\title{COP701 Sample Test cases}
\date{August 2024}
\begin{document}

\section{Introduction to IIT Delhi}
\subsection{Overview}
\subsubsection{History}

\textit{Indian Institute of Technology Delhi (IIT Delhi) is one of the pren
\textbf{Founded in 1961, IIT Delhi has grown into a world-class institution

\hrule

IIT Delhi is located in Hauz Khas, New Delhi. It offers undergraduate, post
The institute is renowned for its rigorous academic programs and distinguis
```

```
\begin{verbatim}
def iit_delhi_info():
    print("Welcome to IIT Delhi!")
    print("Explore the world-class research and academic programs.")
\end{verbatim}

For more information, visit the official IIT Delhi website: \href{https://w

\includegraphics[width=0.5\textwidth]{images/technology.jpg}

\begin{itemize}
    \item Established: 1961
    \item Location: Hauz Khas, New Delhi
    \item Programs: B.Tech, M.Tech, Ph.D., and more
\end{itemize}

\begin{enumerate}
    \item Campus Facilities
    \item Research Opportunities
    \item Alumni Achievements
\end{enumerate}

\begin{tabular}{|c|c|}
\hline
Department & Programs \\
\hline
Computer Science & B.Tech, M.Tech, Ph.D. \\
Electrical Engineering & B.Tech, M.Tech, Ph.D. \\
\hline
\end{tabular}
\end{document}
```

Abstract Syntax Tree File: `ast.tex`

```
document:
  preamble:
    documentclass: \documentclass{article}
    usepackage:
      package: \usepackage{graphicx} % Required for inserting images
      package: \usepackage{hyperref}
    title: \title{COP701 Sample Test cases}
    author:
    date: \date{August 2024}
  body:
    begindocument:
      sections:
        section: Introduction to IIT Delhi
        subsections:
```

```
        subsection: Overview
        subsubsections:
          subsubsection: History
            italic:
              italic_text: Indian Institute of Technology Delhi (IIT Dell
            bold:
              bold_text: Founded in 1961, IIT Delhi has grown into a worl
            hrule:
              hrule_text: ---
            para:
              para_text: IIT Delhi is located in Hauz Khas, New Delhi. It
            para:
              para_text: The institute is renowned for its rigorous acade
            verbatim:
              start_verbatim: ```python
              verbatim_body:
                verbatim_text: def iit_delhi_info():
                verbatim_text:     print("Welcome to IIT Delhi!")
                verbatim_text:     print("Explore the world-class researc
              end_verbatim: ```
            para:
              para_text: For more information, visit the official IIT De
            graphics:
              graphics_text: \includegraphics[width=0.5\textwidth]{images
            itemize:
              start_itemize:
              itemize_body:
                itemize_text:     \item Established: 1961
                itemize_text:     \item Location: Hauz Khas, New Delhi
                itemize_text:     \item Programs: B.Tech, M.Tech, Ph.D.,
              end_itemize:
            enumerate:
              start_enumerate:
              enumerate_body:
                enumerate_text:     \item Campus Facilities
                enumerate_text:     \item Research Opportunities
                enumerate_text:     \item Alumni Achievements
              end_enumerate:
            table_content:
              table_head_block:
                table_text: Department & Programs \\
              hline:
              table_body_block:
                table_text: Computer Science & B.Tech, M.Tech, Ph.D. \\
                table_text: Electrical Engineering & B.Tech, M.Tech, Ph.D
    enddocument:
```

Markdown Output File: `output.md`

```
# Introduction to IIT Delhi
## Overview
### History

*Indian Institute of Technology Delhi (IIT Delhi) is one of the premier eng
**Founded in 1961, IIT Delhi has grown into a world-class institution known

---

IIT Delhi is located in Hauz Khas, New Delhi. It offers undergraduate, post


The institute is renowned for its rigorous academic programs and distinguis

```python
def iit_delhi_info():
    print("Welcome to IIT Delhi!")
    print("Explore the world-class research and academic programs.")
```

For more information, visit the official IIT Delhi website: [IIT Delhi Off

![IIT Delhi Campus](images/technology.jpg)

- Established: 1961
- Location: Hauz Khas, New Delhi
- Programs: B.Tech, M.Tech, Ph.D., and more

1.Campus Facilities
2.Research Opportunities
3.Alumni Achievements

Department	Programs
Computer Science	B.Tech, M.Tech, Ph.D.
Electrical Engineering	B.Tech, M.Tech, Ph.D.
```

0.8.3 Screenshots

Some screenshots of the output and documentation:

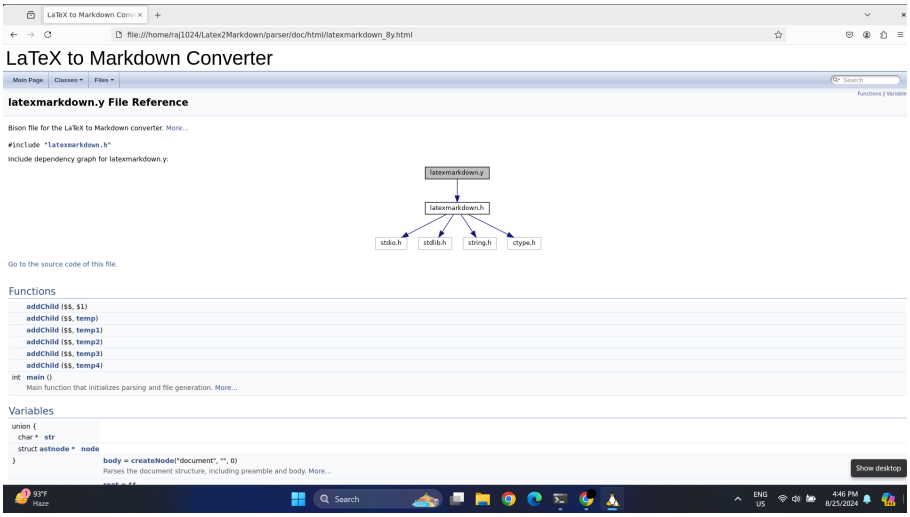

Figure 1: Doxygen Documentation

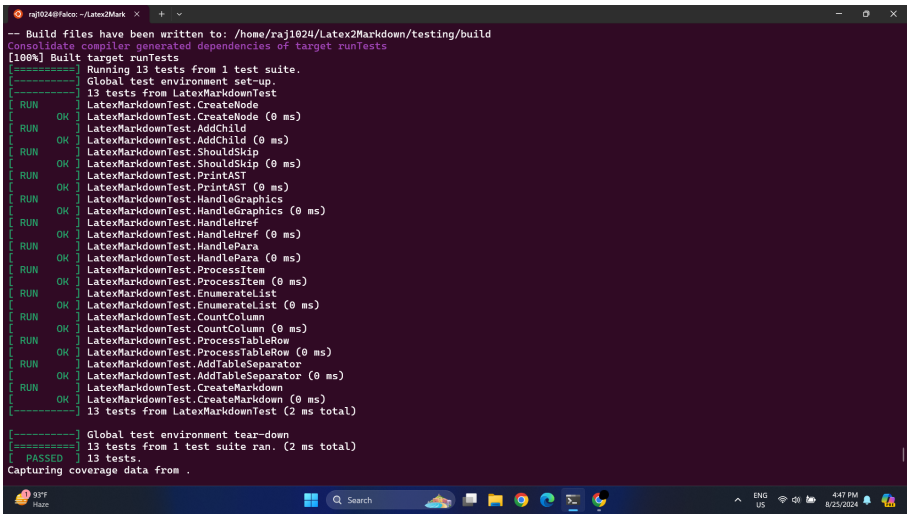

Figure 2: Unit testing

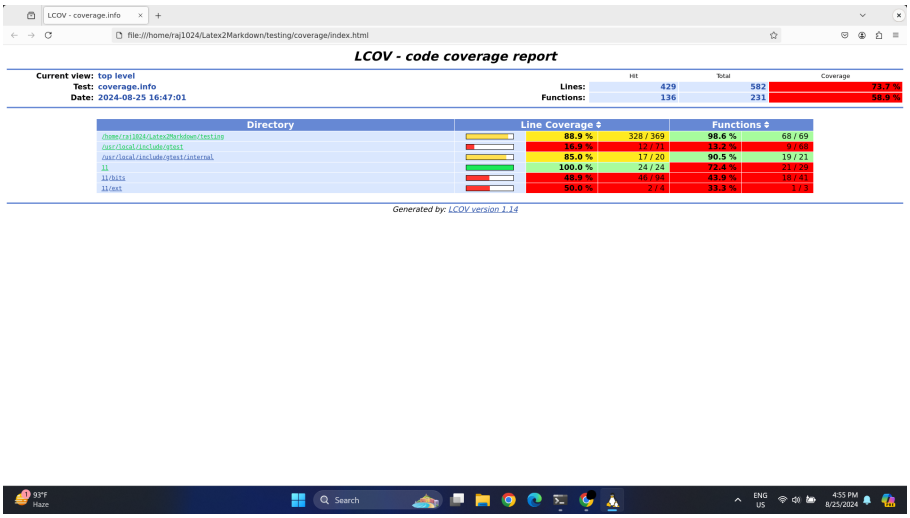

Figure 3: Code coverage - lcov